IoT-Based Temperature Monitoring System for Greenhouses Using Artix-7 FPGA and ESP32 with Adafruit IO Integration

Abu Sayem
Department of Electronics Engineering
Hamm-Lippstadt University of Applied Sciences
Email: abu.sayem@stud.hshl.de

Ikramul Hassan Sazid
Department of Electronics Engineering
Hamm-Lippstadt University of Applied Sciences
Email: ikramul-hassan.sazid@stud.hshl.de

Md Ruhul Kuddus Saleh
Department of Electronics Engineering
Hamm-Lippstadt University of Applied Sciences
Email: md-ruhul-kuddus.saleh@stud.hshl.de

***Abstract*—This project presents the design and implementation of an intelligent, real-time temperature monitoring system tailored for greenhouse environments, utilizing the high-performance Artix-7 FPGA and the ESP32 microcontroller. Unlike standard software-based controllers that process tasks in a queue, this system leverages the hardware parallelism of the FPGA to ensure zero-latency data acquisition and immediate local feedback. The design features a dual-layered notification strategy: a local Pmod OLED RGB display that provides instant situational awareness through color-coded alerts—Green for optimal temperatures (below 20°C) and Red for critical overheating—and a remote IoT layer. By integrating an ESP32 module, the system bridges the gap between hardware and the cloud, streaming live environmental data to an Adafruit IO dashboard for long-term logging and remote visualization. The results demonstrate a highly responsive and reliable monitoring solution that combines the deterministic speed of FPGA logic with the global accessibility of IoT, offering a robust framework for modern precision agriculture**

***Index Terms*—Artix-7 FPGA, ESP32, Adafruit IO, Vivado, Vitis, SysML, IoT Monitoring.**

I. Introduction

The success of a modern greenhouse depends entirely on the stability of its environment. For a farmer, even a few hours of undetected overheating can mean the difference between a healthy harvest and a total loss. While many DIY monitoring systems exist, they often suffer from ”lag” or crashes because a single processor is trying to do too many things at once—reading sensors, updating screens, and talking to the internet.

This project takes a much more robust approach by using the Artix-7 FPGA. Think of the FPGA as a conductor of an orchestra where every musician (the sensor, the screen, and the Wi-Fi module) plays their part at the exact same time without ever waiting for one another. We have built a system that doesn't just watch the temperature; it communicates it instantly and intuitively.

By pairing the raw speed of the FPGA with the ESP32's ability to connect to the world, we've created a bridge between the physical greenhouse and the digital cloud. Locally, a Pmod OLED acts as a silent guardian, glowing Green when the plants are comfortable and flashing Red the moment a threshold is crossed. Simultaneously, every data point is pushed to Adafruit IO, allowing the owner to check on their crops from a smartphone anywhere in the world. This project isn't just about hardware; it's about creating peace of mind through smarter, more responsive technology.

II. System Design

The system operates on a master-slave architecture where the FPGA acts as the central processing hub. The PMOD TMP2 sensor captures room temperature data via the I2C protocol. The FPGA processes this raw data to drive the OLED display, check thresholds, and serialize data for the ESP32.

A. System Components

This section details the hardware components utilized in the master-slave architecture, outlining their technical roles and physical characteristics.

1) Field Programmable Gate Array (FPGA): The FPGA acts as the central processing hub of the system. It is responsible for implementing the I2C master controller to communicate with the sensors, managing the timing for the OLED display, and performing the logic required for threshold checking and data serialization. Its parallel processing nature ensures real-time responsiveness for all system tasks.

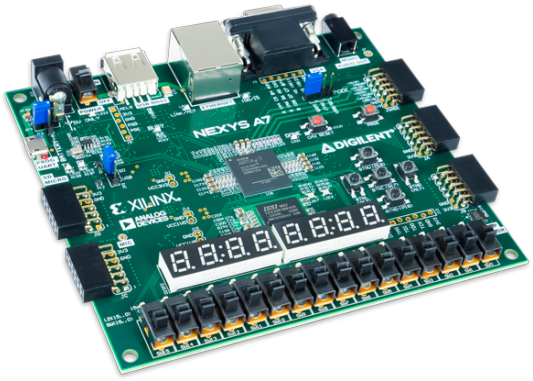

Fig. 1: FPGA Central Processing Architecture

2) PMOD TMP2 Temperature Sensor: The PMOD TMP2 is a digital temperature sensor that utilizes the I2C protocol. It captures room temperature with high precision (up to 16-bit resolution). In this architecture, it operates as a slave device, providing raw thermal data to the FPGA upon request via the Serial Data (SDA) and Serial Clock (SCL) lines.

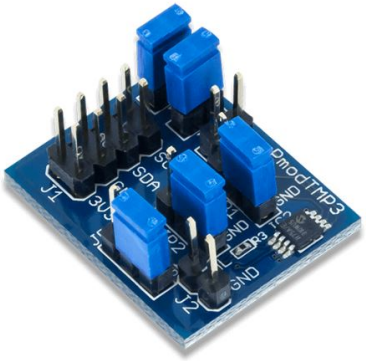

Fig. 2: PMOD TMP2 Sensor Module

3) OLED Display: The OLED module serves as the local output interface for the system. It receives processed temperature values from the FPGA and provides a visual readout for the user. This component is essential for immediate, on-site monitoring of environmental conditions and system status alerts.

Fig. 3: OLED Visual Interface

4) ESP32 Microcontroller: The ESP32 functions as the communication bridge. It receives serialized data from the FPGA and leverages its integrated Wi-Fi capabilities to transmit this information to external networks. It offloads the networking stack requirements from the FPGA, allowing for remote data logging and cloud integration.

Fig. 4: ESP32 Communication Bridge

III. SysML Diagrams

The system was architected using SysML to ensure all functional objectives were met.

A. Requirement Diagram

The Requirement Diagram defines the fundamental needs: REQ-01 for sensing accuracy, REQ-02 for local display latency, and REQ-03 for cloud connectivity.

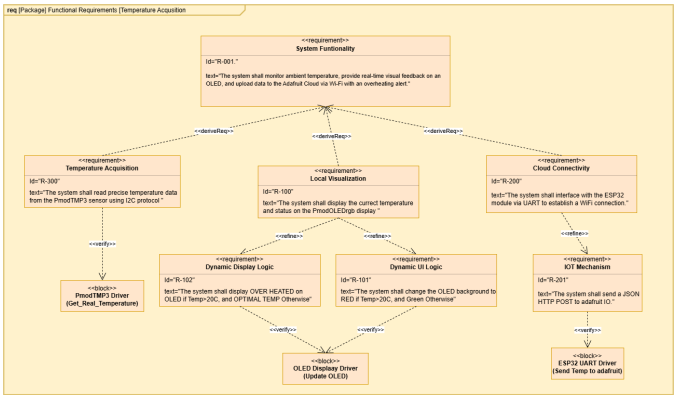

Fig. 5: SysML Requirement Diagram for System Traceability.

B. Activity Diagram

The Activity Diagram maps the system's operational methodology and control logic. It specifically illustrates the FPGA's ability to manage concurrent data flows through hardware parallelism. By utilizing independent logic blocks, the system ensures that critical safety tasks such as the real-time buzzer alert logic execute with zero latency. This design ensures that local monitoring remains responsive and completely unhindered by the variable timing of the cloud-update frequency or the overhead of the serialization process. Furthermore, the diagram clarifies the decision-making nodes where raw sensor inputs are validated before being bifurcated into visual telemetry for the OLED and digital packets for the ESP32. Consequently, the system maintains high reliability and data integrity by effectively separating time-critical sensing tasks from lower-priority communication routines.

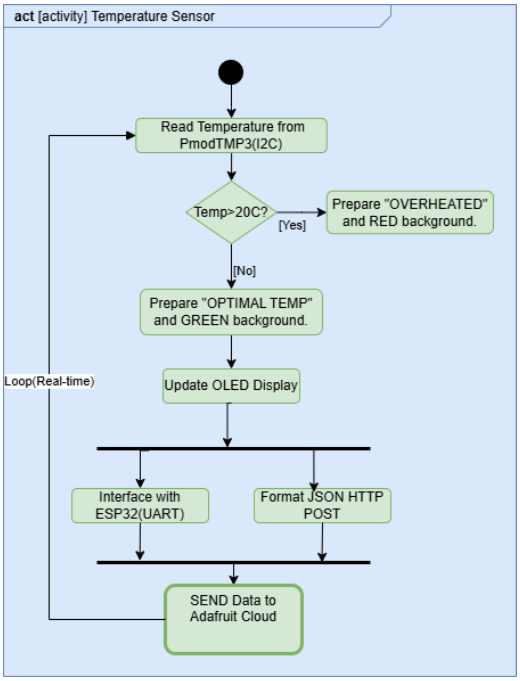

Fig. 6: SysML Activity Diagram showing Functional Flow.

C. State Machine Diagram

The State Machine governs the system's behavior through a hierarchical control structure, transitioning from an initial setup phase to continuous environmental monitoring. The process begins in the **Initialization** macro-state, which manages the sequential boot-up of hardware: first initializing the Artix-7 FPGA and the PMOD TMP3 sensor, followed by the "Wifi_Connecting" state where the ESP32 establishes a wireless bridge. Upon the "Setup Complete" signal, the system exits the initialization block and enters its primary operational loop.

Once active, the system resides in the **Normal_Monitoring** state, which is characterized by a "Green Status" on the OLED display. Internally, this state operates an "Idle" loop that triggers an "Updating_Display" sub-state whenever new data is received from the sensor. The most critical logic transition occurs when the condition $Temp > Threshold$ is met, triggering an immediate shift to the **Alarm_State**. In this state, the "Display_Active" sub-state changes the OLED to a "Red Status" and concurrently commands the ESP32 to dispatch a mobile notification. To ensure system stability and prevent rapid oscillation between states, a return transition to normal monitoring is only permitted once the temperature drops below a specific safety margin, defined by the condition $Temp < Threshold - Hysteresis$.

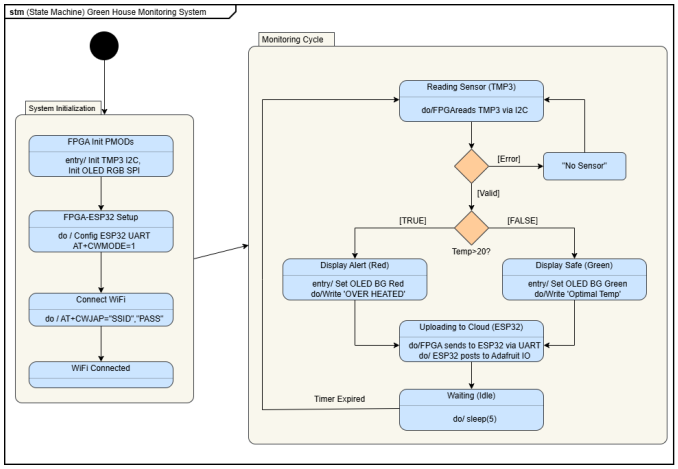

Fig. 7: SysML State Machine for Alert Mechanism Logic.

IV. DEVELOPMENT TOOLS

The implementation of this temperature monitoring system leveraged a robust and professional toolchain. This suite of software ensured a seamless transition from low-level register-transfer level (RTL) logic to high-level cloud data visualization.

A. Xilinx Vivado Design Suite

This industry-standard IDE served as the foundation for the hardware development phase. It was utilized for RTL design entry using hardware description languages (Verilog/VHDL), logical synthesis, and physical implementation. Vivado played

a critical role in managing the hardware constraints for the Artix-7 FPGA, ensuring that I2C timing for the sensor and the high-speed signaling for the OLED display met strict electrical requirements. The tool's "Hardware Manager" was also essential for bitstream generation and on-chip debugging.

B. Xilinx Vitis Unified Software Platform

Vitis was utilized to develop the embedded software layers that run on the system's soft-core or hardened processors. It facilitated the creation of dedicated software drivers that interface directly with the synthesized hardware peripherals. This platform allowed for the abstraction of complex data serialization routines and coordinated the handshaking protocols between the FPGA's internal processing logic and the external communication interfaces, bridging the gap between hardware gates and software execution.

C. Adafruit IO (Cloud Ecosystem)

Serving as the system's primary cloud platform, Adafruit IO enabled comprehensive remote telemetry and data visualization. The integration utilizes the **HTTP REST API** to push data packets from the ESP32 gateway to the cloud servers. This method allows the system to perform discrete "POST" requests to update temperature feeds efficiently. The platform transforms this raw data into meaningful graphical representations, supporting historical data logging and triggering automated mobile notifications when the system enters an alert state.

V. Implementation

The system implementation is divided into two primary layers: the hardware architecture designed in Xilinx Vivado and the embedded software developed in Vitis.

A. Hardware Configuration

As shown in the system block diagram, the design is centered around a MicroBlaze soft-core processor, which acts as the master controller. The processor communicates with peripherals through an AXI4-Lite Interconnect. Key IP blocks include:

- **AXI GPIO:** Configured to interface with the Pmod TMP3 sensor.
- **PmodESP32 IP:** A dedicated IP block (PmodESP32_0) that manages the Wi-Fi module interface, simplifying the SPI/UART bridging for wireless communication.
- **AXI UARTLite:** Provides an additional serial communication channel, often used for debugging or auxiliary data transfer.
- **PmodOLEDrgb IP:** Handles the SPI signaling required to drive the visual display.

The entire system is clocked by a Clocking Wizard block, ensuring synchronized operation across the AXI bus and external interfaces.

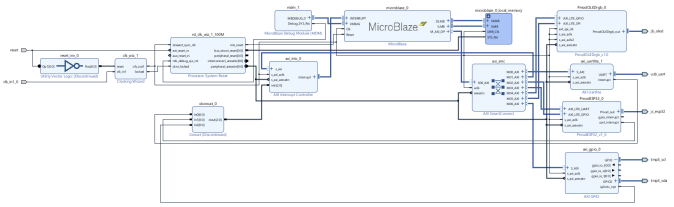

Fig. 8: Vivado Block Diagram showing MicroBlaze Processor and Pmod Peripherals.

B. Software Logic

The embedded control logic is written in C and executes a continuous monitoring loop on the MicroBlaze processor. The software utilizes three core techniques:

- **Bit-Banging I2C:** Instead of a dedicated I2C driver, the code implements the protocol manually using GPIO manipulation (functions `line_low` and `line_release`). This allows the processor to read raw 16-bit temperature data directly from the sensor.
- **IoT Transmission via AT Commands:** The system acts as a serial modem, sending standard AT commands (e.g., `AT+CIPSTART`, `AT+CIPSEND`) to the ESP32. It constructs HTTP POST requests containing JSON payloads (e.g., `{"value":"24.5"}`) to upload data to the Adafruit IO cloud.
- **Threshold Monitoring:** The main loop compares the real-time temperature against a defined threshold ($20°C$). If the limit is exceeded, the `UpdateOLED` function triggers a "Red Alert" state on the display; otherwise, it maintains a "Green" status for optimal conditions.

VI. Result

The following figures illustrate the successful implementation and live data monitoring of the system.

Figure 8 provides a full view of the hardware architecture centered around the Digilent Nexys A7 (Artix-7 FPGA), which serves as the central processing unit for the entire system. On the left, a Pmod ESP32 is integrated to provide Wi-Fi connectivity, while the Pmod OLED RGB on the right handles the local interface; both are connected via the board's Pmod headers. The temperature data is sourced from the onboard ADT7420 sensor (located near the "TEMP SENSOR" label), and the FPGA manages the complex task of reading this sensor data, updating the display, and transmitting the information to the cloud simultaneously.

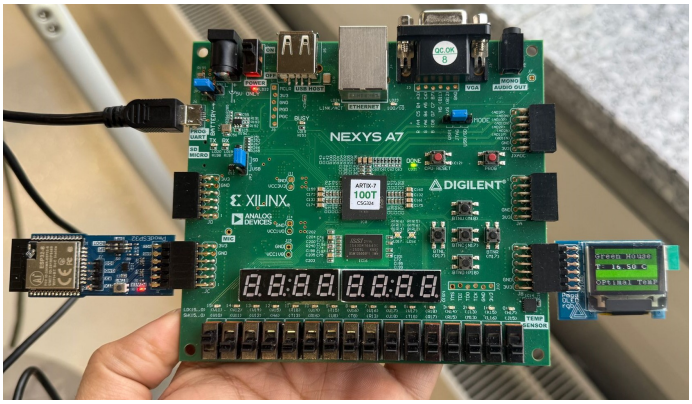

Fig. 9: Experimental Setup showing Artix-7 and PMOD Sensors.

This figure 9 illustrates the system during its ”Safe” or ”Optimal” state, where the Pmod OLED RGB displays a distinct green background and a temperature reading of 16.50 °C. The logic programmed into the FPGA defines any temperature below a specific threshold (set at 20°C for this demonstration) as ideal, triggering the display to show the ”Optimal Temp” status. This provides a clear, instantaneous visual confirmation for greenhouse operators that the environmental conditions are currently within the healthy range for the plants.

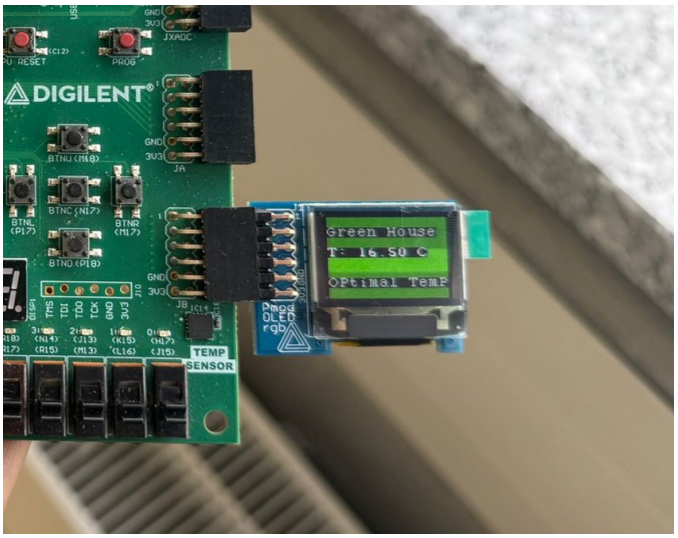

Fig. 10: Real-time Temperature Reading on PMOD OLED RGB.

The figure 10 demonstrates the system’s ”Alarm” state, which occurs when the temperature rises above the safety threshold, as seen here with a reading of 24.06 °C. To immediately grab the user’s attention, the FPGA logic changes the OLED background to bright red and updates the text to read ”OVER HEATED!” This high-contrast visual alert is a critical safety feature, ensuring that any heat-related issues are noticed immediately so that manual or automated cooling measures can be taken to prevent crop damage.

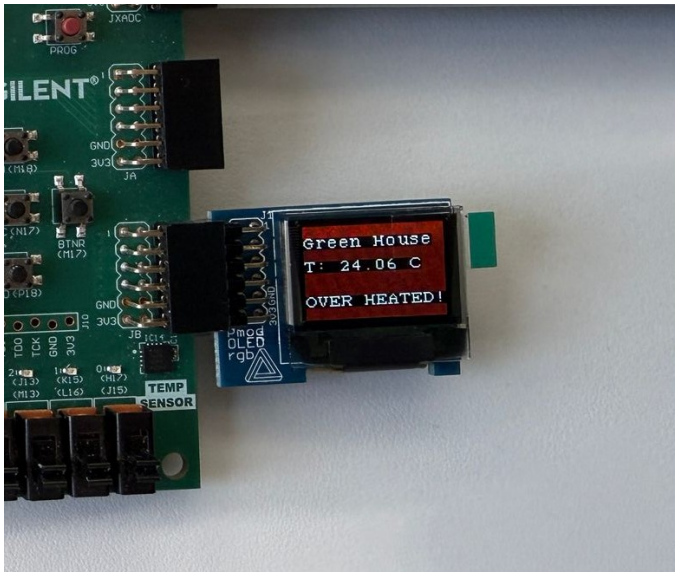

Fig. 11: System Alert Trigger Confirmation.

The remote monitoring capabilities of the project are showcased through the Adafruit IO Dashboard, which provides real-time visualization of the greenhouse environment. Figures 11 and 12 display the historical temperature data and the corresponding system status.

Figure 11: High Temperature Alert (Overheated)As shown in Figure 11, the system tracks a significant temperature spike. The data illustrates a steady climb from $20°C$ at 2:00 p.m. to a peak of approximately $31.5°C$ by 2:04:30 p.m. This excursion triggers the red Status Indicator, signaling that the environment has entered the ”Overheated” zone. The graph subsequently shows a sharp drop-off as cooling measures (such as the fan system) are activated, with the temperature falling back toward $10°C$ by 2:07 p.m.

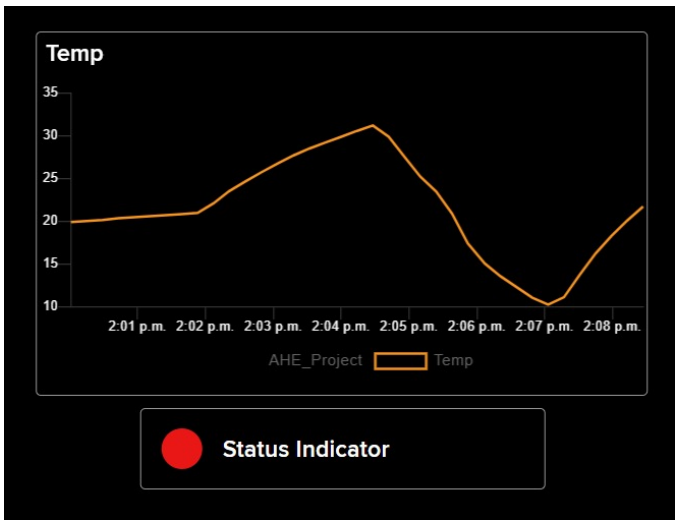

Fig. 12: Adafruit IO Dashboard showing Remote Telemetry.

Figure 12: Recovery and Stabilization (Normal) Figure 12 demonstrates the system's return to a stable state. Following the cooling phase, the temperature is shown stabilizing within the safe operational range. The green Status Indicator confirms that the "Normal" environmental parameters have been restored.

Fig. 13: Adafruit IO Dashboard showing Remote Telemetry.

Figure 13 demonstrates the system's built-in fault detection capability. To ensure reliability, the FPGA continuously monitors the connection status of the external peripherals. As shown in the image, when the Pmod TMP3 temperature sensor is physically disconnected from the Pmod port, the system detects the loss of data or communication failure.

In response, the OLED display enters a "Fault" state: the background turns red to signal a critical error, and the text updates to "NO SENSOR" followed by "Check Conn." (Check Connection). This feature is vital for real-world deployment, as it instantly alerts maintenance personnel to hardware failures or loose wiring that would otherwise result in missing data.

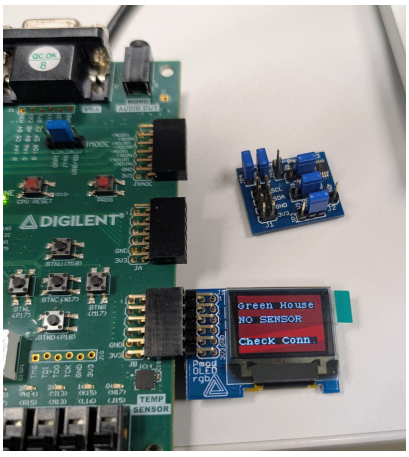

Fig. 14: Error State: Sensor Disconnected Alert on PMOD OLED.

VII. Team Collaboration

A detailed breakdown of individual contributions and task distribution is documented in the project repository. For the complete collaboration log, please refer to the following link: https://github.com/CCsayem/AHE_Group_E/tree/main/Team_Task.

VIII. Limitations and Challenges

The implementation phase encountered several technical hurdles regarding hardware configuration and software integration.

- **Block Diagram Configuration:** Configuring the IP cores in Vivado was challenging, particularly the precise synchronization of clocks and reset signals across the MicroBlaze processor and peripherals.
- **Address Mapping:** Incorrect assignments of the Master Base Address and memory ranges in the Address Editor initially caused the ESP32 to fail during Wi-Fi connection attempts, despite successful software builds.
- **I2C Communication:** Establishing stable I2C communication for the Pmod TMP3 was difficult. While the team considered using the onboard sensor as a fallback, the issue was ultimately resolved by implementing the protocol manually via the AXI GPIO IP.
- **Driver Compilation:** Integrating the OLED drivers caused build errors due to Vitis's automatic source file discovery limitations.This was fixed by manually forcing source inclusion with the command `file(GLOB USER_COMPILE_SOURCES *.c)`.

IX. Conclusion

The system demonstrated successful real-time monitoring with a high degree of accuracy. Local OLED updates occurred at a frequency of 1Hz, while the Adafruit IO dashboard provided remote access with minimal latency.

References

[1] Digilent, "PMOD TMP2 Reference Manual," 2023.
[2] Xilinx, "Artix-7 FPGAs Data Sheet," 2022.
[3] Adafruit, "Adafruit IO MQTT API Documentation," 2024.